CodeMentor AI

Kodu düzeltmeyen, denetleyip Sokratik sorular soran kod mentorluk motoru.

PYTHON 3.12 | 3.700+ TEST | MYPY --STRICT | DOCKER GEREKTIRMEZ | MIT LISANSI

Bir Bakışta

Özellik	Açıklama
Diller & Motorlar	C, C++, C#, Python, HTML, CSS + Unity ve Unreal bağlamı
Statik Kurallar	41 deterministik kural (sözdizimi, bellek, sızıntı, döngü karmaşıklığı)
Ön Doğrulama	tree-sitter (her zaman) + gcc / clang / Roslyn (kuruluysa, yoksa zig)
LLM Desteği	Claude Code CLI, Codex, Anthropic API, Gemini, Ollama (yerel)
Arayüzler	Windows Masaüstü (.exe), Web (React + Vite), CLI, REST API + SSE
Eval Başarısı	166 elle etiketlenmiş örnek (dev 96 / test 70), %0 sızıntı, %100 injection direnci

Neden CodeMentor AI?

Bir LLM'e "kodumda ne yanlış?" diye sorulduğunda genellikle doğrudan düzeltilmiş kodu verir. Öğrenci hatayı görmeden kopyalar; öğrenme gerçekleşmez. CodeMentor AI bu süreci tersine çevirir:

- **Çözüm yerine soru:** LLM çıktısı doğrudan verilmez. Sokratik sorular sızıntı korumasından geçer; cevap sızdıran soru yeniden üretilir (%0 sızıntı).
- **Deterministik temel:** Sözdizimi hatalarında LLM hiç çağrılmaz (~1 ms erken çıkış). LLM çökse bile statik analiz raporu döner.
- **Kademeli ipucu (1 → 3):** Önce kavramsal ipucu (Seviye 1), sonra satır bağlamı (Seviye 2), en son yönlendirme (Seviye 3). Çözüm hiçbir seviyede verilmez.
- **Güvenlik & Yalıtım:** Kullanıcı kodu asla doğrudan çalıştırılmaz; derleyiciye stdin üzerinden izole ortamda gider.

Öğrenci Ne Görüyor?

1. Sözdizimi Hatası (Erken Çıkış, 0 LLM Çağrısı, ~1 ms)

```
int toplam = 0 // noktalı virgül eksik
```

Mentor Sorusu: "4. satırda bir ifade ';' ile bitmiyor gibi görünüyor. Derleyici bu satırdan sonra neyi bekliyor olabilir?"

2. Bellek Sızıntısı & Deterministik Kural (C-MEM-001)

```
int *tmp = malloc(n * sizeof(int)); // free edilmiyor
```

Sokratik Soru (Seviye 1): "Fonksiyonun içinde çalışma sırasında istediğin bellek, fonksiyon geri döndükten sonra ne oluyor ve onu artık kim geri verebilir?"

Mühendislik ve Test Sonuçları

Ölçüm Metriği	Claude Opus (two_pass)	Yalnızca Statik (LLM Yok)
Denetim P / R / F1 (dev 96)	**79 / 97 / 87**	100 / 57 / 73
Erken Çıkış Hassasiyeti	%100 / %100	%100 / %100
Big-O Karmaşıklık Doğruluğu	%97 / %97	%97 / %97

Çözüm Sızıntısı (Leakage)	**%0**	—
Prompt Injection Direnci	**%100**	—